1、phase 1

（1）关键汇编：

```
402036:       sub    $0x10,%rsp
40203a:       call   402c00 <valid_u32_fields>
40203f:       test   %al,%al
402041:       jne    402050 <phase_01+0x30>
402043:       add    $0x10,%rsp
402047:       xor    %eax,%eax
402049:       pop    %rbx
40204a:       pop    %r12
40204c:       pop    %rbp
40204d:       ret
40204e:       xchg   %ax,%ax
402050:       xor    %eax,%eax
402052:       lea    -0x14(%rbp),%rdx
402056:       mov    $0x403182,%esi
40205b:       mov    %rbx,%rdi
40205e:       call   4011e0 <_init+0x1e0>
402063:       cmp    $0x1,%eax
402066:       jne    402043 <phase_01+0x23>
402068:       mov    %r12,%rdi
40206b:       call   401fe0 <p1_compute>
402070:       cmp    %eax,-0x14(%rbp)
402073:       sete   %al
```

```
401fe8:       mov    %rdi,%rdx
401feb:       mov    %edi,%r8d
401fee:       shr    $0x20,%rax
401ff2:       shr    $0x10,%rdx
401ff6:       mov    %eax,%r9d
401ff9:       mov    %eax,%esi
401ffb:       mov    %rsp,%rbp
401ffe:       call   401fb0 <p1_step0>
402003:       mov    %rdx,%rsi
402006:       mov    %r9d,%edi
402009:       mov    %eax,%ecx
40200b:       call   401fc0 <p1_step1>
402010:       mov    %r8d,%esi
402013:       mov    %eax,%edi
402015:       call   401fd0 <p1_step2>
40201a:       pop    %rbp
40201b:       xor    %ecx,%eax
40201d:       ret
```

```
yangxinyuan@localhost:~/Lab2$ objdump -d ./bomb | grep -A 15 "<p1_step0>:"
0000000000401fb0 <p1_step0>:
  401fb0:       f3 0f 1e fa             endbr64
  401fb4:       8d 86 04 29 00 00       lea    0x2904(%rsi),%eax
  401fba:       31 f8                   xor    %edi,%eax
```

```
0000000000401fc0 <p1_step1>:
  401fc0:       f3 0f 1e fa             endbr64
  401fc4:       81 f6 01 07 00 00       xor    $0x701,%esi
  401fca:       8d 04 7e                lea    (%rsi,%rdi,2),%eax
  401fcd:       c3                      ret
```

```
yangxinyuan@localhost:~/Lab2$ objdump -d ./bomb | grep -A 15 "<p1_step2>:"
0000000000401fd0 <p1_step2>:
  401fd0:       f3 0f 1e fa             endbr64
  401fd4:       8d 04 bf                lea    (%rdi,%rdi,4),%eax
  401fd7:       81 c6 4c d9 00 00       add    $0xd94c,%esi
  401fdd:       31 f0                   xor    %esi,%eax
```

（2）计算关系：

```
lo=(uint32_t)challenge;
hi=(uint32_t)(challenge>>32);
mid=(uint32_t)(challenge>>16);
s0=(hi+0x2904u) ^lo;
s1=(mid^0x701u)+2u*hi;
s2=5u*s1^(lo+0xd94cu);
answer=s2^s0;
```

（3）推理过程：

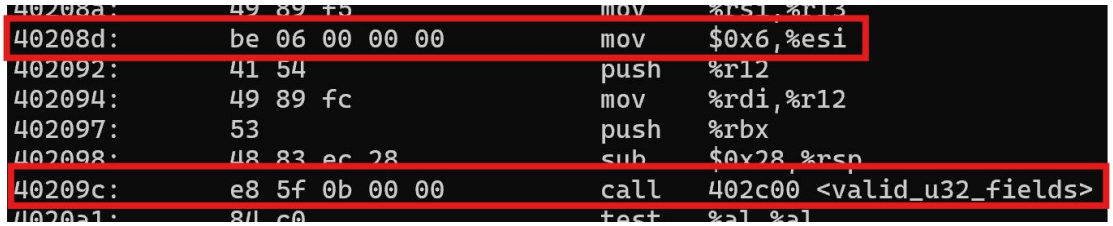
入口处 rdi 仍为 challenge，edi 是低 32 位 lo；shr$0x20 得到高 32 位 hi，shr $0x10 得到中间 mid。step0 把 hi 加 0x2904 后与 lo 异或；step1 把 mid 异或 0x701 后再加 2*hi；step2 先把 step1 结果乘 5，再与(lo+0xd94c)异或；最后把 step2 结果与 step0 结果异或就是期望值。三个函数都只写 eax/esi/edi，因此全部按 32 位无符号计算。

（4）验证的记录：

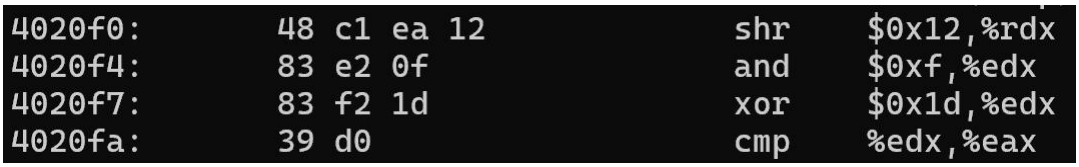


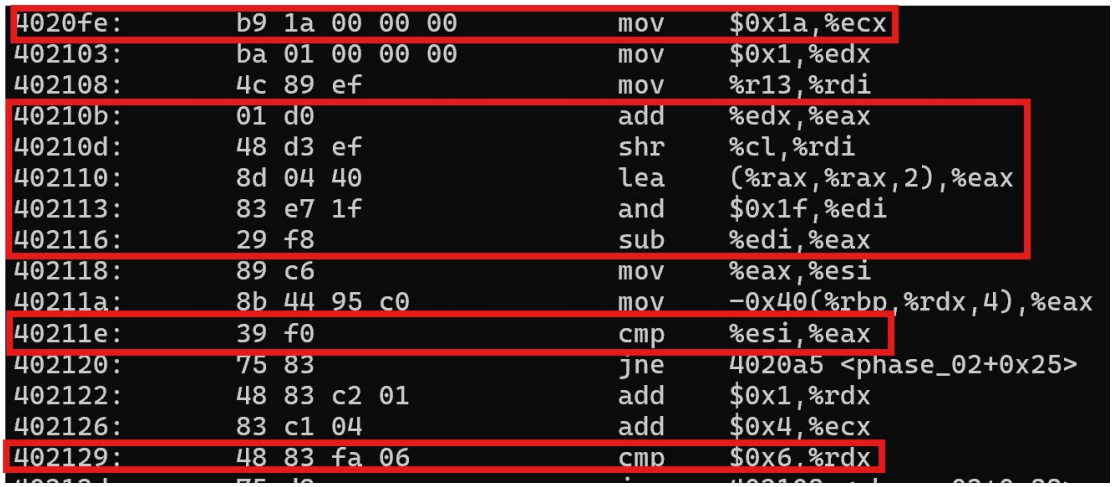
2、phase 2

（1）关键汇编：









（2）计算关系：

a0=((uint32_t)(challenge >> 18) & 0xfu)^0x1du;

for i=1..5:

a_i=3u * (a_{i-1} + i)-((uint32_t)(challenge>>(26 + 4*(i-1))) &0x1fu);

（3）推理过程：

先单独看数组首元素的比较：challenge 右移 18 位、取低 4 位再异或 0x1d，就 a0。进入循环后，寄存器里保存的是上一项结果，先加上序号 i,lea (%rax,%rax,2) 即 3*rax，再减去 challenge 移位后取出的低 5 位；每轮移位计数加 4，直到 i=6 停止。由此得到六阶递推。

（4）验证记录：

```
yangxinyuan@localhost:~/Lab2$ ./defuser --seed 725 --phase 2 | ./bomb --seed 725 --phase 2
BOMBLAB 2026                                    这里还没有明天

                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x0b0b16951ded8ada
Input (unsigned decimal integers):
Phase 2 defused.
```

3、phase 3

（1）关键汇编：

```
40224d:       be 02 00 00 00          mov    $0x2,%esi
402252:       53                      push   %rbx
402253:       48 89 fb                mov    %rdi,%rbx
402256:       48 83 ec 10             sub    $0x10,%rsp
40225a:       e8 a1 09 00 00          call   402c00 <valid_u32_fields>
```

```
402291:       83 e7 07                and    $0x7,%edi
402294:       39 7d e8                cmp    %edi,-0x18(%rbp)
```

```
4022a9:       e8 32 ff ff ff          call   4021e0 <p3_dispatch>
4022ae:       39 45 ec                cmp    %eax,-0x14(%rbp)
4022b1:       0f 94 c0                sete   %al
```

```
0000000000402140 <p3_case_0>:
  402140:       f3 0f 1e fa             endbr64
  402144:       8d 84 bf b6 b5 02 00    lea    0x2b5b6(%rdi,%rdi,4),%eax
  40214b:       c3                      ret
  40214c:       0f 1f 40 00             nopl   0x0(%rax)

0000000000402150 <p3_case_1>:
  402150:       f3 0f 1e fa             endbr64
  402154:       89 f8                   mov    %edi,%eax
  402156:       d1 ef                   shr    %edi
  402158:       35 a5 ff 00 00          xor    $0xffa5,%eax
  40215d:       01 f8                   add    %edi,%eax
  40215f:       c3                      ret
```

```
00000000004021a0 <p3_case_6>:
  4021a0:       f3 0f 1e fa             endbr64
  4021a4:       8d 04 bd 00 00 00 00    lea    0x0(,%rdi,4),%eax
  4021ab:       35 e8 6d 00 00          xor    $0x6de8,%eax
  4021b0:       c3                      ret
```

（2）计算关系：（表下标=challenge&7）

x=(uint32_t)(challenge>>32)^(uint32_t)(challenge>>3);

0:5*x+0x2b5b6　　1:(x^0x7401)+(x>>2)

2:5*x+0x854c　　3:(4*x)^0x6de8

4:5*x+0x4fd03　　5:(2*x)^3

6:3*x+0x3685　　7:(x^0xffa5)+(x>>1)

（3）推理过程：

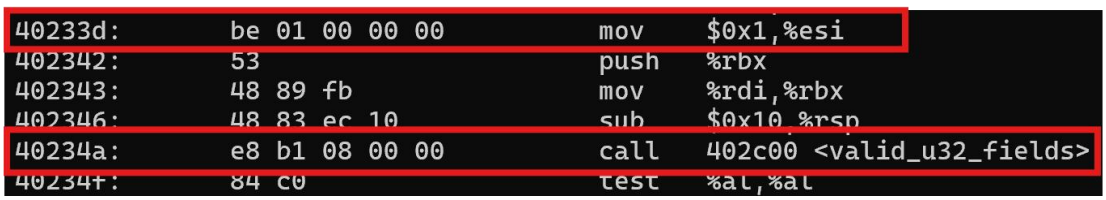
第一个数直接由 challenge 低位给出,不需要计算。第二个数先构造 x:高 32 与(challenge>>3) 的低 32 位异或;再按 challenge&7 选分支。逐个读 case 函数:case0 是 5x+常数,case1 是(x^常数)+(x>>2),case2 是 5x+常数,case3 是 4x^常数,case4 是 5x+常数,case5 是 2x^3,case6 是 3x+常数,case7 是(x^常数)+(x>>1)。注意跳转表下标不等于 case 编号,必须按表项地址对应。

（4）验证记录：



4、phase 4

（1）关键汇编：



```
40236e:       e8 6d ee ff ff          call   4011e0 <_init+0x1e0>
402373:       83 f8 01                cmp    $0x1,%eax
402376:       75 db                   jne    402353 <phase_04+0x23>
```

```
402395:       e8 26 ff ff ff          call   4022c0 <p4_walk>
40239a:       39 45 ec                cmp    %eax,-0x14(%rbp)
```

```
402320:       8d 47 56                lea    0x56(%rdi),%eax
402323:       31 c8                   xor    %ecx,%eax
```

```
40230a:       8d 04 40                lea    (%rax,%rax,2),%eax
40230d:       31 d8                   xor    %ebx,%eax
40230f:       48 8b 5d f8             mov    -0x8(%rbp),%rbx
402313:       c9                      leave
402314:       83 f0 19                xor    $0x19,%eax
```

```
4022ec:       8d 04 40                lea    (%rax,%rax,2),%eax
4022ef:       31 d8                   xor    %ebx,%eax
4022f1:       48 8b 5d f8             mov    -0x8(%rbp),%rbx
4022f5:       c9                      leave
4022f6:       83 f0 08                xor    $0x8,%eax
```

（2）计算关系：

Target=(uint32_t)challenge&0x7f;

key=((uint32_t)(challenge>>7))^(uint32_t) (challenge>>32);

base:(mid + 0x56u)^key

right: (3u*sub^(mid+key))^0x8u

left:(3u*sub^(mid+key))^0x19u

（3）推理过程：

这是一个在[0,127]上查找 target 的二分递归：每层先取中点，比较 target 与中点决定走左还是右，节点值 mid+key 在回代时参与运算。基例命中时返回 (mid+0x56)^key。

（4）验证记录：



5、phase 5

（1）关键汇编：

```
40241a:       49 89 f4                 mov    %rsi,%r12
40241d:       be 02 00 00 00           mov    $0x2,%esi
402422:       53                       push   %rbx
402423:       48 89 fb                 mov    %rdi,%rbx
402426:       48 83 ec 10              sub    $0x10,%rsp
40242a:       e8 d1 07 00 00           call   402c00 <valid_u32_fields>
40242f:       84 c0                    test   %al,%al
402452:       e8 89 ed ff ff           call   4011e0 <_init+0x1e0>
402457:       83 f8 02                 cmp    $0x2,%eax
40245a:       75 d7                    jne    402433 <phase_05+0x23>
40245c:       48 8d 75 ec              lea    -0x14(%rbp),%rsi
402460:       4c 89 e7                 mov    %r12,%rdi
402463:       e8 48 ff ff ff           call   4023b0 <p5_traverse>
402468:       89 c2                    mov    %eax,%edx
40246a:       31 c0                    xor    %eax,%eax
40246c:       39 55 e4                 cmp    %edx,-0x1c(%rbp)
40246f:       75 c4                    jne    402435 <phase_05+0x25>
402471:       8b 45 ec                 mov    -0x14(%rbp),%eax
402474:       39 45 e8                 cmp    %eax,-0x18(%rbp)
402477:       0f 94 c0                 sete   %al
```

```
4023bd:       83 e0 07                 and    $0x7,%eax
4023c0:       48 c1 e9 03              shr    $0x3,%rcx
4023c4:       48 8b 14 c5 20 5b 40     mov    0x405b20(,%rax,8),%rdx
4023cb:       00
4023cc:       48 89 f8                 mov    %rdi,%rax
4023cf:       48 c1 ef 20              shr    $0x20,%rdi
4023d3:       48 c1 e8 06              shr    $0x6,%rax
4023d7:       31 f8                    xor    %edi,%eax
4023d9:       8b 72 04                 mov    0x4(%rdx),%esi
4023dc:       89 cf                    mov    %ecx,%edi
4023de:       35 63 3a 00 00           xor    $0x3a63,%eax
4023e3:       03 02                    add    (%rdx),%eax
4023e5:       8d 04 80                 lea    (%rax,%rax,4),%eax
4023e8:       41 89 30                 mov    %esi,(%r8)
4023eb:       31 f0                    xor    %esi,%eax
```

```
4023f7:       48 8b 52 08              mov    0x8(%rdx),%rdx
4023fb:       83 c1 01                 add    $0x1,%ecx
4023fe:       8b 72 04                 mov    0x4(%rdx),%esi
402401:       03 02                    add    (%rdx),%eax
402403:       8d 04 80                 lea    (%rax,%rax,4),%eax
402406:       41 89 30                 mov    %esi,(%r8)
402409:       31 f0                    xor    %esi,%eax
40240b:       39 cf                    cmp    %ecx,%edi
40240d:       73 e8                    jae    4023f7 <p5_traverse+0x47>
```

（2）计算关系：

acc=(((uint32_t)(challenge>>6))^high32)^0x3a63u;

起点=起点表[challenge & 7],沿 next 前进 hops 步;

每节点:acc=5u*(acc + f0)^f1;第二个口令=末节点 f1;

（3）推理过程：

节点结构为{u32 f0;u32 f1;struct *next;},0x405b20 是 8 个起点指针的数组。先用 challenge&7 选起点,初始累加值由(challenge>>6)^high32 再异或 0x3a63 得到;每经过一个节点做 acc=5*(acc+f0)^f1,并把当前 f1 写回输出位置，前进步数 hops=(challenge>>3)&3。

（4）验证记录：

```
yangxinyuan@localhost:~/Lab2$ ./defuser --seed 725 --phase 5 | ./bomb --seed 725 --phase 5
BOMBLAB 2026                                        这里还没有明天

                                *
                                 \
                              .-"""-.
                            .'       '.
                           /    LOOP    \
                          |               |
                           \             /
                            '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xa92280e627d94251
Input (unsigned decimal integers):
Phase 5 defused.
```

6、phase 6

（1）关键汇编：

```
40256d:       be 01 00 00 00          mov    $0x1,%esi
402572:       53                      push   %rbx
402573:       48 89 fb                mov    %rdi,%rbx
402576:       48 83 ec 10             sub    $0x10,%rsp
40257a:       e8 81 06 00 00          call   402c00 <valid_u32_fields>
40257f:       84 c0                   test   %al,%al
```

```
40259e:       e8 3d ec ff ff          call   4011e0 <_init+0x1e0>
4025a3:       83 f8 01                cmp    $0x1,%eax
4025a6:       75 db                   jne    402583 <phase_06+0x23>
4025a8:       4c 89 e7                mov    %r12,%rdi
4025ab:       e8 60 ff ff ff          call   402510 <p6_dispatch>
```

```
40251c:       48 c1 ef 06              shr    $0x6,%rdi
402520:       48 89 d8                 mov    %rbx,%rax
402523:       48 c1 e8 20              shr    $0x20,%rax
402527:       31 c7                    xor    %eax,%edi
402529:       48 89 d8                 mov    %rbx,%rax
40252c:       48 83 ec 08              sub    $0x8,%rsp
402530:       83 e3 38                 and    $0x38,%ebx
402533:       83 e0 07                 and    $0x7,%eax
402536:       ff 14 c5 c0 5d 40 00     call   *0x405dc0(,%rax,8)
40253d:       35 21 9f 00 00           xor    $0x9f21,%eax
402542:       89 c7                    mov    %eax,%edi
402544:       ff 93 c0 5d 40 00        call   *0x405dc0(%rbx)
40254a:       48 8b 5d f8              mov    -0x8(%rbp),%rbx
40254e:       c9                       leave
40254f:       05 10 48 00 00           add    $0x4810,%eax
402554:       c3                       ret
```

（2）计算关系：

v=((uint32_t)(challenge>>6))^(uint32_t) (challenge>>32);

a=op[challenge & 7](v)^0x9f21u;

answer=op[((uint32_t)(challenge>>3))&7](a)+0x4810u;

表序列={0,5,2,3,4,1,2,3}

（3）推理过程：

两级间接调用共用同一张函数指针表 0x405dc0。先算 v=(challenge>>6)^high32，用 challenge&7 选第一个 op；结果异或 0x9f21 后，再用(challenge>>3)&7 选第二个 op，最后加 0x4810。从表里读出下标序列{0,5,2,3,4,1,2,3}，再逐个反汇编六个 op，再用 switch 复刻

（4）验证记录：



7、secret

（1）关键汇编：

```
40271b:       81 bd 50 ff ff ff 4f    cmpl   $0x4e45504f,-0xb0(%rbp)
402722:       50 45 4e
402725:       75 d4                   jne    4026fb <secret_gate+0x4b>
402727:       4c 89 f2                mov    %r14,%rdx
40272a:       4c 89 ee                mov    %r13,%rsi
40272d:       48 8d bd 54 ff ff ff    lea    -0xac(%rbp),%rdi
402734:       e8 97 fe ff ff          call   4025d0 <secret_phase>
```

```
4025e2:       be 03 00 00 00          mov    $0x3,%esi
4025e7:       53                      push   %rbx
4025e8:       48 89 fb                mov    %rdi,%rbx
```

```
402626:       e8 b5 eb ff ff          call   4011e0 <_init+0x1e0>
40262b:       83 f8 03                cmp    $0x3,%eax
40262e:       75 c8                   jne    4025f8 <secret_phase+0x28>
402630:       44 89 e9                mov    %r13d,%ecx
402633:       31 d2                   xor    %edx,%edx
402635:       83 e1 03                and    $0x3,%ecx
402638:       39 4d d4                cmp    %ecx,-0x2c(%rbp)
```

```
402640:       44 69 c9 29 f3 00 00    imul   $0xf329,%ecx,%r9d
```

```
402664:       e8 57 ff ff ff          call   4025c0 <secret_mix>
402669:       44 31 c8                xor    %r9d,%eax
```

```
402671:       c1 ee 05                shr    $0x5,%esi
402674:       41 8d 94 08 1e 9b 00    lea    0x9b1e(%r8,%rcx,1),%edx
40267b:       00
40267c:       01 f2                   add    %esi,%edx
40267e:       31 d0                   xor    %edx,%eax
```

```
4025c6:       c1 c8 05                ror    $0x5,%eax
4025c9:       35 37 68 00 00          xor    $0x6837,%eax
```

（2）计算关系：

k=context&3;

s1=(challenge>>32)^(uint32_t)challenge;

s2=(context>>32)^(uint32_t)context;

m=ror32(s1 ^ s2, 5)^0x6837u;

t=m^((k*0xf329u)^((uint32_t)(context>>16)));

u=t^(s2+k+0x9b1eu+(s1>>5));

输出：32 个空格+“OPEN k t u”

（3）推理过程：

第一个数就是 context&3。secret_mix 是循环右移 5 位再异或 0x6837;t 等于 mix 结果异或(k*0xf329^context>>16);u 再异或(s2+k+0x9b1e+s1>>5)。前 32

字节填充是为了让“OPEN”落在 gate 检查的偏移处,随后的数字由 sscanf 解析,前导空格会被自动跳过。

（4）验证记录：

```
yangxinyuan@localhost:~/Lab2$ { ./defuser --seed 725; ./defuser --secret --seed 725; } | ./bomb --seed 725
```

```
Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

7、公开测试截图

```
Public tests: 60/60
```

```
Secret public tests: 10/10 (optional)
```

8、辅助工具用途

（1）AI 在 phase 1 的时候辅助我理解了题目的要求

（2）在我编程出错的时候纠正我的错误